

Advertiser Pricing & Free-Tier Revenue Share (2026)

Authoritative summary of the current advertiser pricing. Admin-tunable via settings; nothing here is a promise of returns. Not legal advice — the revenue-share is counsel-gated.

Reconciled 2026-08-15; re-checked against code 2026-09-04; updated 2026-09-11 (fixed two citation drifts: the upgrade-discount setting is `0.055`, not `0.06`, and `FOUNDING_SIGNUP_CREDIT_USD` is retired to `0`). Consistent with the other Get Goods Gratis docs as of this date. The Founding Tier 1 offer, the 5.5% decoupled upgrade discount, the \$2,000 premium gift boost, Tier 2 pay-as-you-go, the discontinued free earn-to-unlock tier, and the three OFF-by-default credit products all match the current Get Goods Gratis documents. There is now a public `/Apply` page that markets this pricing — Founding Tier 1 as the prominent offer, Tier 2 as available, and the three credit products as “coming soon.” See `APPLY-AND-COMING-SOON.md`.

Paid advertiser (PPC / Tier 1)

- **PPC network advertising is included for all three tiers (2026-09-11).** Every advertiser — Tier 1 (Founding), Tier 2 (Scale) and Tier 3 (Unlimited) — can advertise on the premium PPC AdGrid network: their product runs in the AdGrid, users watch it and answer its questions and are credited, and the advertiser pays per engagement.
- **Price: \$12,000 per year, or \$1,000 per month — paid upfront.**
- Settings: `PPC_GRID_ANNUAL_PRICE = 12000`, `PPC_GRID_MONTHLY_PRICE = 1000`, `FOUNDING_ADVERTISER_PRICE_USD = 12000`, `FOUNDING_ADVERTISER_MONTHLY_PRICE_USD = 1000`.

Founding upgrade discount + premium gift boost

- Pay **\$12,000 upfront**. Founding advertisers then get a **discount on an upgrade** (default “**Tier 2 — Scale,**” **\$200,000**): a promotional **5.5% off** the upgrade price → **\$11,000 off** → **net \$189,000**. A founding Tier 1 member can **claim** it within a **12-month window** after they join — and, unlike the general Tier 1→ Tier 2 rollover (which keeps the 5.5% only through the first year), **founding members keep the 5.5% for life**: it comes off **every Tier 2 part in perpetuity** (`TIER2_FOUNDING_DISCOUNT_PERPETUAL = true`). The discount is defined as a **% of the upgrade price — decoupled from the amount paid** (no “credit,” no “return your \$12k”), which removes the return-of-capital signal the founding packet flagged.
- **Premium gift boost: up to \$2,000 in non-cashable store credit** — a **premium-member benefit**, funded by a **collective advertiser pool** and **decoupled from the price you pay**. Tier 1 includes premium, so Tier 1 members qualify like any premium member; the boost is **not** granted from, tied to, or a return of their \$12,000 fee (no return-of-capital signal, same as the upgrade discount). Members claim it from the pool, **subject to availability**, and choose how much to apply and to which items. Nothing is owed. (Replaces the old \$1,000 vesting credit.) See `PREMIUM-GIFT-BOOST.md` and `PREMIUM-BOOST-ADVERTISING.md`.
- Settings: `FOUNDING_UPGRADE_DISCOUNT_PCT = 0.055` (5.5%), `FOUNDING_UPGRADE_PRICE_USD = 200000`, `FOUNDING_UPGRADE_DISCOUNT_WINDOW_MONTHS = 12` (window to claim), `TIER2_FOUNDING_DISCOUNT_PERPETUAL = true` (founding members keep it for life) / `TIER2_DISCOUNT_FIRST_YEAR_ONLY = true` (non-founding loses it after year one), `FOUNDING_SIGNUP_CREDIT_USD = 0` (**RETIRED** — the sign-up store credit is off; benefit delivered via the premium gift boost instead). Code: `backend/sdk/founding-rollover.ts` (claim window) + `backend/sdk/tier2-scaling.ts` (`tier2DiscountRate` applies the per-part rate), functions `foundingRolloverStatus` / `foundingUpgradeQuote`, page `/FoundingUpgrade`.
- **Note:** for maximum daylight from the return-of-capital signal, set the discount % so its dollar result isn’t exactly \$12,000. The \$200k upgrade must be a real product before it’s sold. Full write-up: **FOUNDING-ROLLOVER-AND-SIGNUP-CREDIT.md**.

Tier 1 “pay-from-earnings” — FINANCED (recourse credit — OFF by default)

- A third way to take Tier 1: **\$0 upfront**, the site **sweeps the advertiser’s in-app earnings toward the \$12,000** over a 12-month term, and **at term end any remaining balance is DUE** (recourse — the \$12,000 is owed regardless of earnings).
- **This is regulated credit** and is **disabled by default**, hard-gated behind a licensed creditor (`TIER1_FINANCED_PROVIDER`) + counsel sign-off (`TIER1_FINANCED_LEGAL_SIGNOFF`) + the `tier1_financed` flag. It is heavier than the non-recourse Goods Advance and must not launch without counsel + licensing.
- Full write-up: **TIER1-FINANCED-PAY-FROM-EARNINGS.md**.

Free earn-to-unlock advertiser — DISCONTINUED

Removed per owner decision — the free earn-to-unlock tier is no longer offered.

`FREE_ADVERTISER_TIER_ENABLED` now defaults to **0 (off)**: the tier is not shown and the join endpoint refuses `free_earn` mode. The code and the settings below are **retained but inactive** (set the flag to 1 to bring it

back). The associated revenue-share only ever applied to free-tier advertisers, so it is moot while the tier is off. The separate **no-upfront Tier 1** option (`TIER1_NOUPFRONT_ENABLED`) is unaffected.

The mechanics below are kept for reference only.

- **(Inactive) Still free. Nobody owes anything. Non-recourse throughout.**
- **Step 1 — earn the ~\$8,000 unlock over the 4-year term** (`TARGET_USER_LTV_USD = 8000` , `FREE_TIER_TERM_YEARS = 4`). This \$8,000 **counts toward parity** with a paid \$12,000 advertiser.
- **Step 2 — revenue-share recovers only the REMAINING \$4,000** (= \$12,000 parity target – \$8,000 earn-to-unlock credit), taken as a share of the advertiser’s **own business revenue**:
 - **10%** of business revenue **until the remaining \$4,000 is recovered**, then
 - **5%** of business revenue **in perpetuity**, for as long as they use the platform.
- Settings: `FREE_ADVERTISER_REVSHARE_TARGET_USD = 12000` , `FREE_ADVERTISER_EARN_UNLOCK_CREDIT_USD = 8000` (⇒ revenue-share recovers \$4,000), `FREE_ADVERTISER_REVSHARE_PCT = 0.10` , `FREE_ADVERTISER_REVSHARE_POST_PCT = 0.05` .
- Logic: `freeAdvertiserRevenueShareCut()` in `backend/sdk/earned-advertiser.ts` applies 10% only up to the \$4,000 remainder (splitting any revenue that crosses the line), then 5% thereafter.

Worked example (business revenue)

Their business revenue	10% cut/mo	Months to clear the \$4,000	5% perpetual tail
\$20,000/mo	\$2,000	2 months	\$1,000/mo
\$6,667/mo	~\$667	6 months	~\$333/mo
\$3,333/mo	~\$333	12 months	~\$167/mo
\$1,667/mo	~\$167	24 months	~\$83/mo

(“Clear \$12,000 in 6 months at 10%” would need ~\$20k/mo of business revenue. Because the \$8,000 earn-to-unlock counts toward parity, the revenue-share only has to recover \$4,000 — so a \$6,667/mo advertiser clears in ~6 months, and a \$20k/mo advertiser clears the \$4,000 in ~2 months.)

Business revenue = platform-attributed sales (the measured, enforceable definition)

Decision: “business revenue” is **platform-attributed sales** — the sales the advertiser actually makes *through the platform*, recorded by us. It is **not** their self-reported total company revenue and **not** any off-platform sales. This is the version we can measure, audit, and enforce without trusting a number the advertiser types in.

How it’s measured in code. The revenue-share reads finalized sales rows the platform already stores, stamped with the advertiser, and sums the ones in a “counted” (paid/fulfilled) status:

- Source is config-driven so the mapping can change without a redeploy. Defaults:
 - `FREE_ADVERTISER_REVSHARE_SOURCE = platform_attributed` (set to `off` to disable the share entirely)
 - `REVSHARE_SALES_ENTITY = Order` — the entity that holds sales
 - `REVSHARE_SALES_ADVERTISER_FIELD = advertiser_id` — the field on that row identifying the advertiser
 - `REVSHARE_SALES_AMOUNT_FIELD = amount` — the sale amount to sum
 - `REVSHARE_SALES_COUNTED_STATUSES = awaiting_shipment,shipped,delivered,fulfilled,completed,paid`
- Logic: `attributedSalesUsd()` and `computeFreeAdvertiserRevenueShare()` in `backend/sdk/earned-advertiser.ts` sum those rows and apply the tiered cut (10% then 5%).
- **Attribution is required, and the default is safe.** A sale only counts toward an advertiser’s business revenue if its row is **stamped with that advertiser’s id** (`advertiser_id`). Orders that aren’t stamped read as **\$0** — the share silently collects nothing rather than guessing. So the one integration step to make this real is: **stamp the advertiser id on Orders that came from that advertiser’s placement** (their storefront, sponsored listing, or attributed click). Until orders carry that field, the free tier costs the advertiser nothing.

Keep it a revenue-share, not a loan

- **Non-recourse.** Taken **only** from platform-attributed sales that actually occur. No sales → nothing taken, nothing owed. No fixed balance, no debt, no charge, no collections.
- Measured on **revenue** (verifiable platform sales), never “profit,” and never a self-reported figure.
- For business advertisers this is commercial revenue-based financing (far lighter than consumer credit) — but the moment it becomes a fixed amount owed regardless of sales, it’s a loan. Do not cross that line.
- Have commercial-finance counsel review the revenue-share terms + disclosures before enabling.

Ad-network metrics + demographic & audience targeting (2026-09 update)

Full network-standard metric set (measured, never guaranteed). Advertisers now get the complete mobile-ad-network metric set, computed from real platform activity: **eCPM, CPM, IPM, CPP**, and the windowed **ROAS curve (D1 / D3 / D7 / D14 / D28 / D90 / D365)**, plus the publisher-side yield metrics — **fill rate, ARPDAU and retention (D1 / D7 / D28)**. Every figure is measured and shown with its basis; the platform never guarantees an ROI. (Engine: `ad-metrics.ts`; on-demand read: `adMetricsReport`.)

Tracked over time and AI-optimized. A scheduled sweep (`adMetricsSweep`) tracks each metric as history (`OptimizationSignal` + `AgentLearningMemory` — no new tables) and runs an AI optimizer that biases delivery from measured results. Every automatic action is reversible and routed through the autonomy kernel (`ad_optimization`): it auto-applies on trust or queues a human review, and it **never raises spend and never touches billing or payouts** (those stay permanently gated).

Demographic + new-vs-existing targeting. On top of the interest/behavior cohorts, advertisers can target **demographics — age range, gender, country and region** — and **audience type (new vs existing users)**, matched against users' own non-identifying profile data (never an individual). Advertisers set an `optimize_for` objective — **ROAS, new-user acquisition, or existing-user retention** — that steers the AI delivery optimizer. (Engine: `ad-audience.ts`.)